Emergency Response and Rescue Agent

PROJECT REPORT

Artificial Intelligence

Submitted in partial fulfilment of the requirements for
[Course Code] and [Degree Programme]

Submitted by

[Student Name 1] | [SAP ID 1]

[Student Name 2] | [SAP ID 2]

[Student Name 3] | [SAP ID 3]

[Section] | [Group Number]

Under the supervision of

[Supervisor Name]
[Designation]

[Department or School]
[University or Institute]
[City and State]

[Month and Year]

Declaration and Abstract

Declaration

We declare that the project report entitled Emergency Response and Rescue Agent presents our project work for the Artificial Intelligence course. Ideas, tools and external material used in the report are acknowledged in the bibliography. The report describes an educational simulation and does not claim field deployment or measured improvements in real emergency response.

Student name	SAP ID	Signature and date
[Student Name 1]	[SAP ID 1]	[Signature] [Date]
[Student Name 2]	[SAP ID 2]	[Signature] [Date]
[Student Name 3]	[SAP ID 3]	[Signature] [Date]

Abstract

Emergency response requires decisions about both the route to an incident and the vehicle that should serve it. This project demonstrates those decisions through a browser based simulator of central Dehradun. A real OpenStreetMap road network provides the spatial model, while traffic, road closures and emergency vehicle movement are simulated. The working sector covers approximately 7 km by 7 km.

The system implements Breadth First Search, Depth First Search, Dijkstra's algorithm and A* search. Users can compare route distance, estimated travel time, explored nodes and search runtime. A centralized dispatcher manages a cooperative fleet of ambulances, a fire brigade and a police unit. It orders requests by priority and arrival sequence, then selects the available compatible unit with the lowest planned estimated arrival time.

Manual incidents and seed controlled automatic congestion create a changing environment. Active missions respond to these changes through route replanning or waiting when a road is impassable. A separate manual journey mode explains one vehicle's route in detail. The project demonstrates graph search, cost based decision making, priority scheduling and multi-agent coordination. Its travel estimates and safety scores are simulation assumptions rather than verified operational guidance.

Keywords: artificial intelligence, intelligent agents, graph search, emergency dispatch, A* search, dynamic routing, multi-agent simulation.

Contents

Section	Title	Page
1	Project Description	4
2	System Requirements	5
3	Intelligent Agent Model	6
4	System Architecture	7
5	Search Algorithms	8
6	Data Structures and Route Costs	9
7	Fleet Dispatch and Traffic	10
8	Methods and Implementation	11
9	System Output	12
10	Testing and Result Analysis	13
11	Limitations and Conclusion	14
	Bibliography	15

List of Figures

Figure 1 System workflow 7

Figure 2 Route planning interface 12

Project Identification

Project title: Emergency Response and Rescue Agent
Project number: [Project Number]
Course: Artificial Intelligence
Course code: [Course Code]
Academic session: [Academic Session]

1 Project Description

1 1 Purpose and Background

An emergency vehicle needs a reachable destination, a suitable road route and an assignment that reflects the urgency of the request. The geographically closest vehicle may have a longer journey because of one way streets, congestion or road closures. This project brings route planning and vehicle assignment into one interactive environment so that these decisions can be observed and compared.

The application represents drivable roads in central Dehradun as a directed graph. It uses Python for planning and simulation and a browser interface for map interaction. Users choose a destination, vehicle type, urgency, search algorithm and routing objective. The interface displays the search process and the resulting journey, making abstract AI concepts visible through a practical problem.

1 2 Problem Statement

The problem is to assign available emergency resources to prioritized requests and find passable routes through a changing road network. The system must respect road direction, distinguish physical distance from estimated travel time and respond coherently when incidents invalidate a plan. It must also explain why different search methods can produce different routes or explore different numbers of nodes.

1 3 Objectives

The objectives are to model real road connectivity; implement and compare four graph search algorithms; support distance, time, traffic and safety proxy objectives; coordinate compatible emergency units using priority and ETA; simulate concurrent journeys; and demonstrate how closures and congestion affect planning. Clear displays of route metrics and mission status support evaluation and classroom presentation.

1 4 Scope

The current working sector is approximately 49 square kilometres, centred near latitude 30.3165 and longitude 78.0322. The cached road graph contains 8,877 nodes and 21,010 directed edges. A larger surrounding road layer provides visual context only. The project covers medical, fire and police response routing; it does not model rescue equipment operation, clinical triage, hospital capacity or actual emergency service communication.

2 System Requirements

2 1 Software Requirements

Component	Role in the project
Python 3.12	Validated local runtime for the current Windows setup.
Flask	Serves the dashboard and handles routing, incidents and simulation requests [1].
OSMnx	Retrieves and models the OpenStreetMap drivable road network [2].
NetworkX	Stores and processes the directed road graph [3].
HTML CSS and JavaScript	Provide dashboard controls, state updates and animated results.
Leaflet	Displays interactive maps and vehicle markers [4].
Modern web browser	Runs the interface; JavaScript must be enabled.
Windows 10 or 11	Local demonstration platform; the project is launched from its Python environment.

2 2 Hardware and Connectivity

A standard laptop or desktop is suitable for demonstration. A practical recommended configuration is a modern dual core processor, at least 8 GB RAM, and sufficient free storage for the Python environment, map caches and project files. These are suggested demonstration resources rather than measured minimum requirements. A mouse or touchpad helps place waypoints and incidents precisely.

Internet access is needed when road data must be downloaded and when external map tiles or browser libraries are fetched. The cached graph avoids repeated road network downloads. Cached routing data alone does not guarantee that every browser asset will be available offline.

2 3 Functional Requirements

The system must accept valid in-sector positions, snap them to real road nodes, return a route or an explicit no-route state, and report distance and estimated travel time. It must support algorithm comparison, incident placement and removal, a prioritized request queue, compatible unit assignment, vehicle movement, pause and resume, cancellation and fleet reset. Resource editing must prevent an assigned vehicle from being moved or removed.

2 4 Quality Requirements

Road changes must be consistent across viewers. Delayed responses must not replace newer state. Planning must preserve the base road data, and vehicle movement must follow road geometry. A blocked road must remain impassable for every routing objective. The visual distinction between an algorithm replay and an active mission must remain clear.

3 Intelligent Agent Model

3 1 States Actions and Goal

An agent's state contains its location, vehicle type, availability, assigned request, route, progress and pause or waiting status. The shared state adds queued requests, known incidents and simulation time. Initially, the default units are idle at simulated depots, the request queue is empty and automatic traffic is stopped.

Actions include assigning a request, moving along an adjacent directed road, waiting, replanning, pausing, resuming, cancelling and completing a mission. A mission succeeds when its vehicle reaches the assigned destination after snapping to the road network. Arrival releases the unit for another compatible request. The model is a search and scheduling system; it does not learn policies from training data.

3 2 PEAS Description

Element	Project interpretation
Performance	Priority adherence, compatible ETA, waiting time, travel time, distance, completed requests and replanning behaviour.
Environment	Drivable roads, available vehicles, emergency requests, congestion, accidents and closures.
Actuators	Dispatch decisions, simulated road movement, waiting and route changes.
Sensors	Known graph data, shared incident records, request priorities and simulated unit positions.

3 3 Environment Classification

The simulated state is fully observable because the dispatcher can inspect all modeled roads, requests, unit positions and incidents. Decisions are sequential because one assignment changes later availability. The environment is dynamic because road conditions can change during travel. Planning on a fixed snapshot is deterministic, while automatic traffic is stochastic with a reproducible seed.

The fleet is cooperative and uses a centralized dispatcher. Road choices form a discrete graph, while displayed vehicle motion is interpolated along continuous road geometry. These properties describe the simulator's internal world; actual emergency operations would involve uncertain reports and partially observed traffic conditions.

3 4 Decision Policy

Urgency is supplied by the user as Critical, High, Medium or Low. Requests of equal priority follow arrival order. The dispatcher then minimizes planned ETA among available units of the required type. It does not diagnose medical severity or inspect onboard equipment. Active assignments are non-preemptive, so a later critical request does not interrupt an already assigned mission.

4 System Architecture

The application separates user interaction, planning and the shared simulation. The browser sends choices to Flask, which validates the request and uses the road graph, vehicle profile and current incidents to build a route. The same planning logic supports both a manually dispatched journey and fleet assignments.

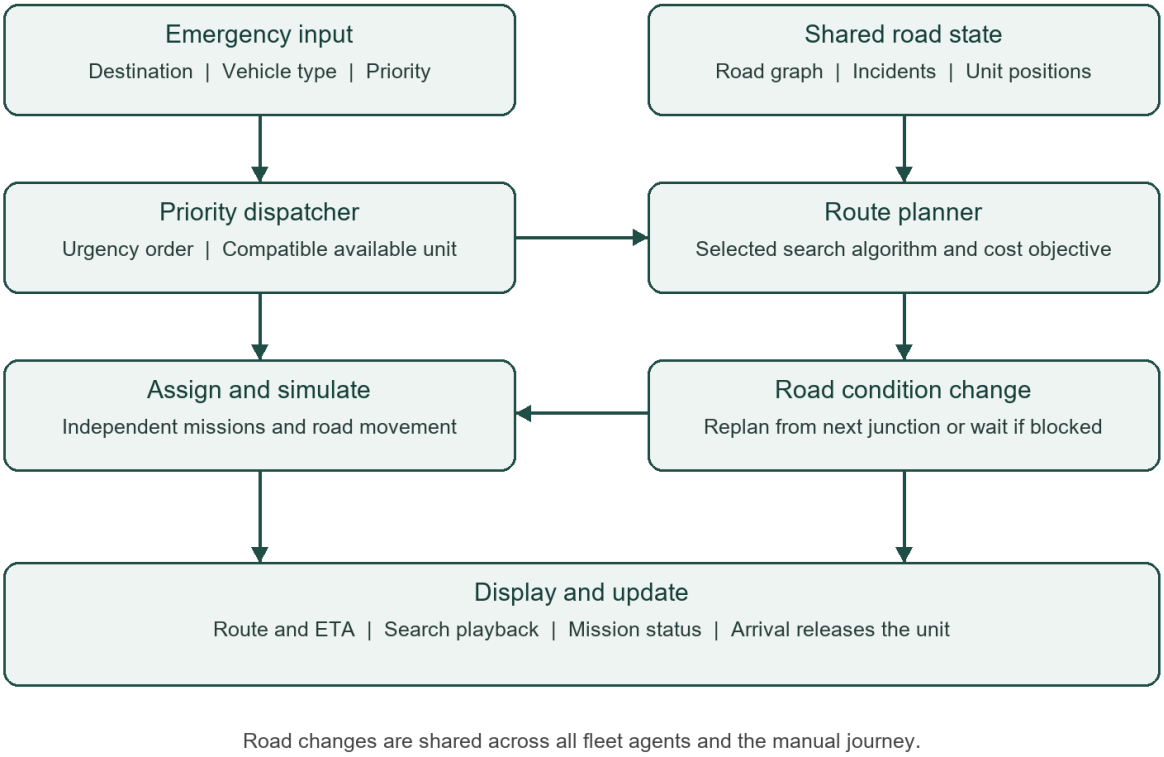


Figure 1 System workflow showing assignment planning and incident feedback

4 1 Main Modules

The interface module handles map selection, controls, metrics and search playback. The graph module loads cached real roads and provides node snapping. The search module implements BFS, DFS, Dijkstra and A*. The vehicle profile module applies type specific speed and road delay assumptions without modifying the base graph.

The incident module records closures and cost multipliers. The fleet module holds requests, unit states, assignment decisions and movement. A separate traffic clock generates congestion. The visualization module draws actual road geometry and replays search or movement states. All fleet viewers observe one shared server world.

4 2 State Management

Road and fleet operations are serialized so a route is evaluated against coherent conditions. Revision and sequence identifiers help the interface reject obsolete responses. Persistent files hold road caches; missions and incidents are held in memory. The present design requires one application worker because separate workers would maintain different simulation worlds.

5 Search Algorithms

5 1 Breadth First Search and Depth First Search

Breadth First Search explores the graph in layers using a queue. It finds a route with the fewest edges in an unweighted interpretation. An edge represents a road connection, so fewer edges do not necessarily mean fewer metres, fewer turns or lower travel time. BFS provides a useful baseline for explaining the difference between graph depth and weighted route cost.

Depth First Search follows a branch deeply before backtracking. It returns the first reachable path found under its traversal order and does not guarantee a minimum cost route. Its exploration and winding routes are useful for comparison with cost guided methods. Both searches exclude blocked edges and respect graph direction.

5 2 Dijkstra Search

Dijkstra's algorithm expands the frontier entry with the lowest accumulated cost. Costs may represent physical distance, estimated time or a weighted road proxy. With non-negative edge weights, it finds a minimum cost route for the selected objective on the current planning snapshot. It also provides a reference for checking A* route cost on the same graph and settings.

5 3 A Star Search

A* evaluates a candidate using $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated cost and $h(n)$ estimates the remaining cost. The project uses straight line Haversine distance as a lower bound. For time routing, this distance is divided by the selected vehicle's maximum modeled speed. A lower bound helps guide the search without overstating the cost still required.

A* is intended to reach the same optimal objective value as Dijkstra while often examining fewer nodes. The amount of reduction depends on graph shape, endpoints and the objective. Equal cost routes may have different geometry, and no fixed speed improvement is claimed.

Algorithm	Frontier strategy	Guarantee in this project
BFS	First in first out queue	Fewest graph edges.
DFS	Depth oriented stack	A first path when reachable; no cost optimality.
Dijkstra	Lowest accumulated cost	Minimum selected weighted cost.
A*	Accumulated cost plus lower bound	Minimum selected weighted cost with a valid heuristic.

6 Data Structures and Route Costs

6 1 Road Graph and Supporting Structures

The road network is a directed multigraph, represented using NetworkX. Nodes carry geographic positions, and edges carry road length, geometry, classification and available speed information. Directed edges preserve one way travel. Multiple edges between the same pair of nodes preserve alternative road connections rather than merging them into a single link.

Structure	Use
Directed multigraph	Road connectivity with direction and parallel road segments.
Queue and stack	BFS frontier and DFS traversal order.
Minimum priority heap	Dijkstra and A* frontier selection.
Sets	Visited nodes and blocked edge membership.
Dictionaries	Best costs, parent links, incidents and unit records.
Request records	Priority, sequence, destination, vehicle type and assignment state.
Bounded motion history	Road samples for smooth display between simulation updates.

6 2 Cost Objectives

Minimum distance sums physical edge lengths. Congestion multipliers do not inflate this objective, although road closures still exclude an edge. Minimum time estimates traversal duration from length and modeled speed, then applies vehicle specific road delays and incident penalties. Route metrics report physical distance separately from estimated time.

The minimum traffic objective applies road class penalties that favour assumed traffic flow. The safety objective applies a different road class weighting. These are explanatory proxies; they are not measured traffic density or validated accident risk. Vehicle delay assumptions and incident multipliers also influence these weighted objectives.

6 3 Vehicle Assumptions

Ambulances have a modeled 50 km/h speed cap. Fire vehicles have a 40 km/h cap and greater local and service road delays. Police vehicles have a 55 km/h cap. These values demonstrate type dependent routing and ETA. They do not assert legal speeds, physical clearance or real response performance.

7 Fleet Dispatch and Traffic

7 1 Priority and Assignment

The default fleet contains two ambulances, one fire brigade and one police unit. A request stores its destination, required vehicle type, priority, algorithm and objective. Critical requests precede High, Medium and Low requests; requests within the same priority follow first in first out order. The scheduler considers available compatible units and chooses the one with the lowest planned ETA.

Each unit retains its own route, progress and mission controls. Missions can run concurrently. A paused or waiting unit remains assigned and cannot serve another request. When no suitable unit or passable route exists, the request remains queued for a later retry. Existing assignments are not interrupted by new requests, even when the new request has higher priority.

7 2 Resource Management

Users can add, move or remove available units. Positions are validated and snapped to road nodes inside the working sector. Assigned units are protected from these edits until their missions finish or are cancelled. Fleet reset returns units to their configured depots, including edited depot positions. Restarting the server restores the default fleet configuration.

7 3 Shared Incidents and Replanning

A roadblock makes the affected road impassable. Accident and traffic incidents increase selected costs and estimated travel duration. Conditions are shared across the fleet and manual journey. A vehicle already partway along a directed road retains its position and unfinished road segment when replanning. The new plan starts from its next junction. If its occupied road becomes closed, it waits rather than moving through the closure.

7 4 Automatic Traffic

Automatic traffic can be started, paused or reset independently of fleet movement. At a chosen interval of 5, 10 or 20 seconds, it generates three congestion incidents with multipliers from 1.5 to 3. A seed makes the event sequence reproducible for the same graph after reset. Pausing retains current congestion; resetting removes generated incidents while preserving manually placed hazards.

The server advances the world when requests arrive; it has no continuously running background traffic worker. Dashboard polling normally occurs every 750 milliseconds. Long unobserved intervals are capped at two real seconds to prevent large catch-up jumps. This timing model supports an interactive demonstration rather than an always active service.

8 Methods and Implementation

8 1 Map Preparation and Input Validation

The application first loads its cached drivable road graph or obtains the required road data when the cache is unavailable. It prepares geometry in the correct travel direction and provides the working bounds to the interface. Waypoints and resource positions are checked against these bounds and snapped to road nodes. The surrounding map layer supplies context and does not extend the routable graph.

8 2 Route Planning Procedure

For a manual route, the user selects a start, destination, vehicle type, algorithm and objective. The server validates the choices, prepares a request specific graph view with vehicle attributes, and reads the current road conditions. The chosen search method returns a route, exploration trace and metrics, or reports that the destination is unreachable. The base road attributes remain unchanged for later requests.

The browser replays the completed search through explored roads and then reveals the chosen path. Pause, speed and replay controls affect this visualization. They do not suspend or resume the backend search computation. The route summary follows recorded search progress before presenting final distance and time.

8 3 Manual Journey Procedure

After route discovery finishes, the user may dispatch the selected vehicle. The display tracks distance travelled, remaining distance, estimated arrival, elapsed simulated time and progress. Journey pause, resume and cancellation operate independently of fleet controls. Movement can run at 10, 30 or 60 times the simulation clock speed. These settings accelerate the demonstration rather than changing the route search algorithm.

8 4 Fleet Procedure and Recovery

For fleet use, the selected destination is queued with a priority and required unit type. Starting the dispatcher activates priority assignment and concurrent movement. The interface interpolates authoritative road samples between server updates, preserving corners in the road geometry. Individual controls affect one unit, while global fleet controls affect the shared fleet clock.

When conditions change, obsolete route results are rejected and affected missions replan or wait. Resume checks current conditions before continuing. Unreachable queued requests retry after relevant road or resource changes. The event log exposes assignment, waiting and replanning decisions, allowing the demonstration to explain both successful movement and explicit blocked states.

9 System Output

The route planning interface combines map selection, vehicle settings, algorithm choice and route metrics. Figure 2 shows an existing project output for an ambulance route from the Railway Station to the Clock Tower using A* and the minimum travel time objective. It illustrates how the system communicates its chosen route and search effort.

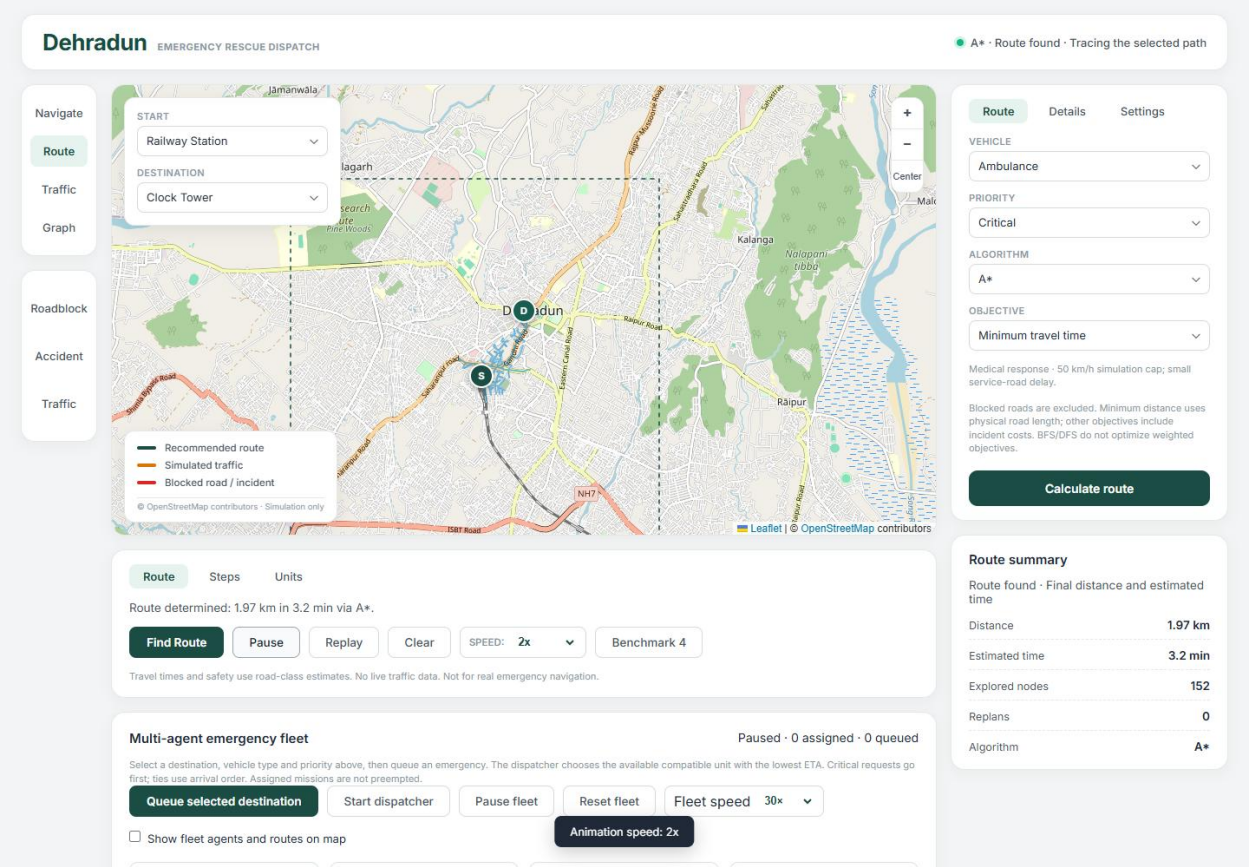


Figure 2 Route planning interface from the project. Map data © OpenStreetMap contributors, available under the Open Database License [5].

9 1 Reading the Output

The pictured run reports 1.97 km, an estimated 3.2 minutes and 152 explored nodes. These values belong to the displayed example and are not a general benchmark. Changing endpoints, incidents, vehicle profile or algorithm can change the result. The estimated time is generated by the model and does not represent a measured ambulance journey.

9 2 Other Observable Results

A comparison panel reports the four algorithms on matching inputs. Fleet cards show assignment and journey status, while the queue shows priority and waiting time. Road closures can produce a new route or a visible waiting state. Arrival completes a mission and makes the unit available for another compatible request. These outputs allow the planner, dispatcher and changing road environment to be assessed together.

10 Testing and Result Analysis

10 1 Verification Approach

Backend verification checks algorithm behaviour, route metrics, road direction, incident handling, dispatcher decisions and request validation. A local verification run on 29 October 2026 completed all 45 backend tests successfully. This result establishes the tested software behaviours; it does not validate simulated travel times against real emergency journeys.

Test area	Behaviour covered by the backend suite
Search correctness	BFS hop behaviour, DFS traversal, weighted route optimality and A* agreement with Dijkstra.
Graph behaviour	One way and parallel edges, route geometry, same-node routes and unreachable destinations.
Road conditions	Closures, penalties, objective differences and vehicle profile effects.
Scheduling	Priority and FIFO ordering, compatible ETA selection and independent mission control.
Replanning	Preserved progress, closure waiting, resume validation and motion history.
Traffic and resources	Seeded traffic, fleet editing, depot reset and zero-resource queues.
Input handling	Invalid coordinates, bounds checks, node snapping and API validation.

10 2 Interpretation of Results

The algorithm design explains why the shortest weighted route differs from a first-found or fewest-edge route. Dijkstra and A* are appropriate when the selected weighted objective must be minimized. BFS and DFS show alternative exploration strategies, but their output must not be described as optimal for physical distance or time. Search runtime and node count are separate measures and can vary between inputs and machines.

Priority scheduling and ETA selection solve different decisions: priority selects which waiting request is considered first, and ETA selects which compatible available unit should serve it. Closures can make a request temporarily unreachable. Retaining it in the queue or waiting safely is a valid simulation result rather than a reason to display an impossible journey.

10 3 Evaluation Boundaries

Browser test scripts also exist for journey controls, fleet movement, map interactions and delayed replies. The test result above refers specifically to the backend suite run for this report. No percentage reduction in response time, real-world rescue success rate or citywide service capacity is claimed. Such measures would require representative incident data and field validation.

11 Limitations and Conclusion

11 1 Current Limitations

The project is an educational simulator for a bounded part of Dehradun. Its road graph is real map data, but incidents and vehicle movement are simulated. Traffic and safety objectives rely on road class assumptions. The system has no live GPS, authenticated emergency reporting, hospital capacity data or integration with emergency control rooms. Priorities are entered by the user rather than inferred by a clinical or incident classification model.

Vehicle speeds and delays do not model physical clearance, signal priority, driver behaviour or local operating rules. The manual journey and fleet are separate workflows rather than one unified resource ledger. In-memory requests and incidents are lost on server restart. One worker is required for a consistent world, and request driven time advancement limits unattended operation. Network dependencies can also affect map display and first-time data loading.

11 2 Future Enhancements

Future work could introduce persistent mission storage, authenticated roles, verified live vehicle locations and validated traffic observations. A shared state service would support multiple server workers. A dedicated simulation clock could remove dependence on viewer polling. Equipment capabilities, hospital destinations and service duration could extend the resource model beyond vehicle type compatibility.

Evaluation should use repeatable endpoint sets and incident scenarios, recording objective cost, runtime, expanded nodes, waiting time and completed requests. Comparing these measures across seeds and loads would show where each algorithm and assignment policy performs well. Field use would require separate data quality, reliability and operational validation.

11 3 Conclusion

Emergency Response and Rescue Agent combines graph search with priority based resource coordination in an understandable interactive system. It connects a directed road model, four search algorithms, vehicle specific estimates, incident handling and concurrent fleet movement. Users can observe how a route is discovered, why a unit is assigned and how the system reacts when a road becomes unusable.

The completed project provides a practical demonstration of intelligent agents making sequential decisions in a changing shared environment. Its value lies in making planning assumptions and decisions visible, while keeping a clear distinction between a tested educational model and real emergency response operations.

Bibliography

The following references identify the principal software and mapping sources used by the project. Web references were accessed on 29 October 2026.

[1] Pallets Projects. Flask Documentation.
https://flask.palletsprojects.com/en/stable/

[2] OSMnx. Official Documentation.
https://osmnx.readthedocs.io/en/stable/

[3] NetworkX Developers. NetworkX Documentation.
https://networkx.org/documentation/stable/

[4] Leaflet. Official project website and documentation.
https://leafletjs.com/

[5] OpenStreetMap contributors. Copyright and License. Map data available under the Open Database License.
https://www.openstreetmap.org/copyright

Project Materials

Emergency Response and Rescue Agent project repository. Project summary, application documentation, search algorithms, vehicle profiles, fleet simulation and automated backend tests. Local project version reviewed on 29 October 2026.

Project interface output. Existing route planning screenshot used in Figure 2. Numerical values in the screenshot are an illustrative recorded output rather than a controlled benchmark dataset.

Attribution

Road network data and map imagery shown in the interface are credited to OpenStreetMap contributors. The workflow in Figure 1 is an original explanatory diagram of the project's implemented planning and dispatch process.